JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class Logger

java.lang.Object
    java.util.logging.Logger

```
public class Logger
extends Object
```

A Logger object is used to log messages for a specific system or application component. Loggers are normally named, using a hierarchical dot-separated namespace. Logger names can be arbitrary strings, but they should normally be based on the package name or class name of the logged component, such as java.net or javax.swing. In addition it is possible to create "anonymous" Loggers that are not stored in the Logger namespace.

Logger objects may be obtained by calls on one of the getLogger factory methods. These will either create a new Logger or return a suitable existing Logger. It is important to note that the Logger returned by one of the `getLogger` factory methods may be garbage collected at any time if a strong reference to the Logger is not kept.

Logging messages will be forwarded to registered Handler objects, which can forward the messages to a variety of destinations, including consoles, files, OS logs, etc.

Each Logger keeps track of a "parent" Logger, which is its nearest existing ancestor in the Logger namespace.

Each Logger has a "Level" associated with it. This reflects a minimum Level that this logger cares about. If a Logger's level is set to `null`, then its effective level is inherited from its parent, which may in turn obtain it recursively from its parent, and so on up the tree.

The log level can be configured based on the properties from the logging configuration file, as described in the description of the LogManager class.

However it may also be dynamically changed by calls on the Logger.setLevel method. If a logger's level is changed the change may also affect child loggers, since any child logger that has `null` as its level will inherit its effective level from its parent.

On each logging call the Logger initially performs a cheap check of the request level (e.g., SEVERE or FINE) against the effective log level of the logger. If the request level is lower than the log level, the logging call returns immediately.

After passing this initial (cheap) test, the Logger will allocate a LogRecord to describe the logging message. It will then call a Filter (if present) to do a more detailed check on whether the record should be published. If that passes it will then publish the LogRecord to its output Handlers. By default, loggers also publish to their parent's Handlers, recursively up the tree.

Each Logger may have a `ResourceBundle` associated with it. The `ResourceBundle` may be specified by name, using the `getLogger(java.lang.String, java.lang.String)` factory method, or by value - using the `setResourceBundle` method. This bundle will be used for localizing logging messages. If a Logger does not have its own `ResourceBundle` or resource bundle name, then it will inherit the `ResourceBundle` or resource bundle name from its parent, recursively up the tree.

Most of the logger output methods take a "msg" argument. This msg argument may be either a raw value or a localization key. During formatting, if the logger has (or inherits) a localization `ResourceBundle` and if the `ResourceBundle` has a mapping for the msg string, then the msg string is replaced by the localized value. Otherwise the original msg string is used. Typically, formatters use java.text.MessageFormat style formatting to format parameters, so for example a format string "{0} {1}" would format two parameters as strings.

A set of methods alternatively take a "msgSupplier" instead of a "msg" argument. These methods take a `Supplier<String>` function which is invoked to construct the desired log message only when the message actually is to be logged based on the effective log level thus eliminating unnecessary message construction. For example, if the developer wants to log system health status for diagnosis, with the String-accepting version, the code would look like:

```
class DiagnosisMessages {
  static String systemHealthStatus() {
    // collect system health information
    ...
  }
}
...
logger.log(Level.FINER, DiagnosisMessages.systemHealthStatus());
```

With the above code, the health status is collected unnecessarily even when the log level FINER is disabled. With the Supplier-accepting version as below, the status will only be collected when the log level FINER is enabled.

```
logger.log(Level.FINER, DiagnosisMessages::systemHealthStatus);
```

When looking for a `ResourceBundle`, the logger will first look at whether a bundle was specified using `setResourceBundle`, and then only whether a resource bundle name was specified through the `getLogger` factory method. If no `ResourceBundle` or no resource bundle name is found, then it will use the nearest `ResourceBundle` or resource bundle name inherited from its parent tree.
When a `ResourceBundle` was inherited or specified through the `setResourceBundle` method, then that `ResourceBundle` will be used. Otherwise if the logger only has or inherited a resource bundle name, then that resource bundle name will be mapped to a `ResourceBundle` object, using the default Locale at the time of logging.
When mapping resource bundle names to `ResourceBundle` objects, the logger will first try to use the Thread's context class loader to map the given resource bundle name to a `ResourceBundle`. If the thread context class loader is `null`, it will try the system class loader instead. If the `ResourceBundle` is still not found, it will use the class loader of the first caller of the `getLogger` factory method.

Formatting (including localization) is the responsibility of the output Handler, which will typically call a Formatter.

Note that formatting need not occur synchronously. It may be delayed until a LogRecord is actually written to an external sink.

The logging methods are grouped in five main categories:

- There are a set of "log" methods that take a log level, a message string, and optionally some parameters to the message string.
- There are a set of "logp" methods (for "log precise") that are like the "log" methods, but also take an explicit source class name and method name.
- There are a set of "logrb" method (for "log with resource bundle") that are like the "logp" method, but also take an explicit resource bundle object for use in localizing the log message.
- There are convenience methods for tracing method entries (the "entering" methods), method returns (the "exiting" methods) and throwing exceptions (the "throwing" methods).
- Finally, there are a set of convenience methods for use in the very simplest cases, when a developer simply wants to log a simple string at a given log level. These methods are named after the standard Level names ("severe", "warning", "info", etc.) and take a single argument, a message string.

For the methods that do not take an explicit source name and method name, the Logging framework will make a "best effort" to determine which class and method called into the logging method. However, it is important to realize that this automatically inferred information may only be approximate (or may even be quite wrong!). Virtual machines are allowed to do extensive optimizations when JITing

and may entirely remove stack frames, making it impossible to reliably locate the calling class and method.

All methods on Logger are multi-thread safe.

Subclassing Information: Note that a LogManager class may provide its own implementation of named Loggers for any point in the namespace. Therefore, any subclasses of Logger (unless they are implemented in conjunction with a new LogManager class) should take care to obtain a Logger instance from the LogManager class and should delegate operations such as "isLoggable" and "log(LogRecord)" to that instance. Note that in order to intercept all logging output, subclasses need only override the log(LogRecord) method. All the other logging methods are implemented as calls on this log(LogRecord) method.

Since:

1.4

Field Summary

Fields

Modifier and Type	Field	Description
static Logger	global	**Deprecated.** Initialization of this field is prone to deadlocks. The field must be initialized by the Logger class initialization which may cause deadlocks with the LogManager class initialization. In such cases two class initialization wait for each other to complete. The preferred way to get the global logger object is via the call `Logger.getGlobal()`. For compatibility with old JDK versions where the `Logger.getGlobal()` is not available use the call `Logger.getLogger(Logger.GLOBAL_LOGGER_NAME)` or `Logger.getLogger("global")`.
static String	GLOBAL_LOGGER_NAME	GLOBAL_LOGGER_NAME is a name for the global logger.

Constructor Summary

Constructors

Modifier	Constructor	Description

protected	**Logger**(**String** name, **String** resourceBundleName)	Protected method to construct a logger for a named subsystem.

Method Summary

Deprecat

Modifier and Type	Method	Description
void	**addHandler**(**Handler** handler)	Add a log Handle messages.
void	**config**(**String** msg)	Log a CONFIG m
void	**config**(**Supplier**<**String**> msgSupplier)	Log a CONFIG m only to be constru level is such that actually be logge
void	**entering**(**String** sourceClass, **String** sourceMethod)	Log a method ent
void	**entering**(**String** sourceClass, **String** sourceMethod, **Object** param1)	Log a method ent parameter.
void	**entering**(**String** sourceClass, **String** sourceMethod, **Object**[] params)	Log a method ent parameters.
void	**exiting**(**String** sourceClass, **String** sourceMethod)	Log a method ret
void	**exiting**(**String** sourceClass, **String** sourceMethod, **Object** result)	Log a method ret object.
void	**fine**(**String** msg)	Log a FINE messa
void	**fine**(**Supplier**<**String**> msgSupplier)	Log a FINE messa to be constructed is such that the m actually be logge
void	**finer**(**String** msg)	Log a FINER mes
void	**finer**(**Supplier**<**String**> msgSupplier)	Log a FINER mes to be constructed is such that the m actually be logge

void	**finest**(**String** msg)	Log a FINEST me
void	**finest**(**Supplier**<**String**> msgSupplier)	Log a FINEST me only to be constru level is such that actually be logge
static **Logger**	**getAnonymousLogger**()	Create an anonyn
static **Logger**	**getAnonymousLogger**(**String** resourceBundleName)	Create an anonyn
Filter	**getFilter**()	Get the current fi
static **Logger**	**getGlobal**()	Return global log name Logger.GLOBAL_
Handler[]	**getHandlers**()	Get the Handlers this logger.
Level	**getLevel**()	Get the log Level specified for this
static **Logger**	**getLogger**(**String** name)	Find or create a l subsystem.
static **Logger**	**getLogger**(**String** name, **String** resourceBundleName)	Find or create a l subsystem.
String	**getName**()	Get the name for
Logger	**getParent**()	Return the paren
ResourceBundle	**getResourceBundle**()	Retrieve the local bundle for this lo
String	**getResourceBundleName**()	Retrieve the local bundle name for t
boolean	**getUseParentHandlers**()	Discover whether is sending its outp logger.
void	**info**(**String** msg)	Log an INFO mes
void	**info**(**Supplier**<**String**> msgSupplier)	Log a INFO mess to be constructed is such that the m actually be logge

boolean	**isLoggable**(**Level** level)	Check if a messag level would actua this logger.
void	**log**(**Level** level, **String** msg)	Log a message, w
void	**log**(**Level** level, **String** msg, **Object** param1)	Log a message, w parameter.
void	**log**(**Level** level, **String** msg, **Object**[] params)	Log a message, w object arguments
void	**log**(**Level** level, **String** msg, **Throwable** thrown)	Log a message, w Throwable inform
void	**log**(**Level** level, **Supplier**<**String**> msgSupplier)	Log a message, w constructed if the such that the mes be logged.
void	**log**(**Level** level, **Throwable** thrown, **Supplier**<**String**> msgSupplier)	Log a lazily const with associated T information.
void	**log**(**LogRecord** record)	Log a LogRecord.
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** msg)	Log a message, sp class and method arguments.
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** msg, **Object** param1)	Log a message, sp class and method object parameter message.
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** msg, **Object**[] params)	Log a message, sp class and method object arguments
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** msg, **Throwable** thrown)	Log a message, sp class and method Throwable inform
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **Supplier**<**String**> msgSupplier)	Log a lazily const specifying source with no argument
void	**logp**(**Level** level, **String** sourceClass, **String** sourceMethod, **Throwable** thrown, **Supplier**<**String**> msgSupplier)	Log a lazily const specifying source

		with associated T information.
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **ResourceBundle** bundle, **String** msg, **Object**... params)	Log a message, sp class, method, an with an optional l parameters.
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **ResourceBundle** bundle, **String** msg, **Throwable** thrown)	Log a message, sp class, method, an with associated T information.
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** bundleName, **String** msg)	**Deprecated.** Use **logrb(java.util java.lang.Strin java.lang.Strin java.util.Resou java.lang.Strin java.lang.Objec**
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** bundleName, **String** msg, **Object** param1)	**Deprecated.** Use **logrb(java.util java.lang.Strin java.lang.Strin java.util.Resou java.lang.Strin java.lang.Objec**
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** bundleName, **String** msg, **Object**[] params)	**Deprecated.** Use **logrb(java.util java.lang.Strin java.lang.Strin java.util.Resou java.lang.Strin java.lang.Objec**
void	**logrb**(**Level** level, **String** sourceClass, **String** sourceMethod, **String** bundleName, **String** msg, **Throwable** thrown)	**Deprecated.** Use **logrb(java.util java.lang.Strin java.lang.Strin java.util.Resou java.lang.Strin java.lang.Throw**

void	**removeHandler**(**Handler** handler)	Remove a log Ha
void	**setFilter**(**Filter** newFilter)	Set a filter to con Logger.
void	**setLevel**(**Level** newLevel)	Set the log level s message levels w logger.
void	**setParent**(**Logger** parent)	Set the parent for
void	**setResourceBundle**(**ResourceBundle** bundle)	Sets a resource b logger.
void	**setUseParentHandlers**(boolean useParentHandlers)	Specify whether t should send its ou Logger.
void	**severe**(**String** msg)	Log a SEVERE m
void	**severe**(**Supplier**<**String**> msgSupplier)	Log a SEVERE m only to be constru level is such that actually be logge
void	**throwing**(**String** sourceClass, **String** sourceMethod, **Throwable** thrown)	Log throwing an
void	**warning**(**String** msg)	Log a WARNING
void	**warning**(**Supplier**<**String**> msgSupplier)	Log a WARNING only to be constru level is such that actually be logge

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

GLOBAL_LOGGER_NAME

```
public static final String GLOBAL_LOGGER_NAME
```

GLOBAL_LOGGER_NAME is a name for the global logger.

Since:

1.6

See Also:

Constant Field Values

global

```
@Deprecated
public static final Logger global
```

Deprecated. *Initialization of this field is prone to deadlocks. The field must be initialized by the Logger class initialization which may cause deadlocks with the LogManager class initialization. In such cases two class initialization wait for each other to complete. The preferred way to get the global logger object is via the call `Logger.getGlobal()`. For compatibility with old JDK versions where the `Logger.getGlobal()` is not available use the call `Logger.getLogger(Logger.GLOBAL_LOGGER_NAME)` or `Logger.getLogger("global")`.*

The "global" Logger object is provided as a convenience to developers who are making casual use of the Logging package. Developers who are making serious use of the logging package (for example in products) should create and use their own Logger objects, with appropriate names, so that logging can be controlled on a suitable per-Logger granularity. Developers also need to keep a strong reference to their Logger objects to prevent them from being garbage collected.

Constructor Detail

Logger

```
protected Logger(String name,
                 String resourceBundleName)
```

Protected method to construct a logger for a named subsystem.

The logger will be initially configured with a null Level and with useParentHandlers set to true.

Parameters:

`name - A name for the logger. This should be a dot-separated name and should normally be based on the package name or class name of the subsystem, such as java.net or javax.swing. It may be null for anonymous Loggers.`

resourceBundleName - name of ResourceBundle to be used for localizing messages for this logger. May be null if none of the messages require localization.

Throws:

MissingResourceException - if the resourceBundleName is non-null and no corresponding resource can be found.

Method Detail

getGlobal

```
public static final Logger getGlobal()
```

Return global logger object with the name Logger.GLOBAL_LOGGER_NAME.

Returns:

global logger object

Since:

1.7

getLogger

```
public static Logger getLogger(String name)
```

Find or create a logger for a named subsystem. If a logger has already been created with the given name it is returned. Otherwise a new logger is created.

If a new logger is created its log level will be configured based on the LogManager configuration and it will configured to also send logging output to its parent's Handlers. It will be registered in the LogManager global namespace.

Note: The LogManager may only retain a weak reference to the newly created Logger. It is important to understand that a previously created Logger with the given name may be garbage collected at any time if there is no strong reference to the Logger. In particular, this means that two back-to-back calls like `getLogger("MyLogger").log(...)` may use different Logger objects named "MyLogger" if there is no strong reference to the Logger named "MyLogger" elsewhere in the program.

Parameters:

name - A name for the logger. This should be a dot-separated name and should normally be based on the package name or class name of the subsystem, such as java.net or javax.swing

Returns:

a suitable Logger

Throws:

NullPointerException - if the name is null.

getLogger

```
public static Logger getLogger(String name,
                               String resourceBundleName)
```

Find or create a logger for a named subsystem. If a logger has already been created with the given name it is returned. Otherwise a new logger is created.

If a new logger is created its log level will be configured based on the LogManager and it will configured to also send logging output to its parent's Handlers. It will be registered in the LogManager global namespace.

Note: The LogManager may only retain a weak reference to the newly created Logger. It is important to understand that a previously created Logger with the given name may be garbage collected at any time if there is no strong reference to the Logger. In particular, this means that two back-to-back calls like `getLogger("MyLogger", ...).log(...)` may use different Logger objects named "MyLogger" if there is no strong reference to the Logger named "MyLogger" elsewhere in the program.

If the named Logger already exists and does not yet have a localization resource bundle then the given resource bundle name is used. If the named Logger already exists and has a different resource bundle name then an IllegalArgumentException is thrown.

Parameters:

name - A name for the logger. This should be a dot-separated name and should normally be based on the package name or class name of the subsystem, such as java.net or javax.swing

resourceBundleName - name of ResourceBundle to be used for localizing messages for this logger. May be null if none of the messages require localization.

Returns:

a suitable Logger

Throws:

MissingResourceException - if the resourceBundleName is non-null and no corresponding resource can be found.

IllegalArgumentException - if the Logger already exists and uses a different resource bundle name; or if resourceBundleName is null but the named logger has a resource bundle set.

NullPointerException - if the name is null.

getAnonymousLogger

```
public static Logger getAnonymousLogger()
```

Create an anonymous Logger. The newly created Logger is not registered in the LogManager namespace. There will be no access checks on updates to the logger.

This factory method is primarily intended for use from applets. Because the resulting Logger is anonymous it can be kept private by the creating class. This removes the need for normal security checks, which in turn allows untrusted applet code to update the control state of the Logger. For example an applet can do a setLevel or an addHandler on an anonymous Logger.

Even although the new logger is anonymous, it is configured to have the root logger ("") as its parent. This means that by default it inherits its effective level and handlers from the root logger. Changing its parent via the setParent method will still require the security permission specified by that method.

Returns:

a newly created private Logger

getAnonymousLogger

```
public static Logger getAnonymousLogger(String resourceBundleName)
```

Create an anonymous Logger. The newly created Logger is not registered in the LogManager namespace. There will be no access checks on updates to the logger.

This factory method is primarily intended for use from applets. Because the resulting Logger is anonymous it can be kept private by the creating class. This removes the need for normal security checks, which in turn allows untrusted applet code to update the control state of the Logger. For example an applet can do a setLevel or an addHandler on an anonymous Logger.

Even although the new logger is anonymous, it is configured to have the root logger ("") as its parent. This means that by default it inherits its effective level and handlers from the root logger. Changing its parent via the setParent method will still require the security permission specified by that method.

Parameters:

resourceBundleName - name of ResourceBundle to be used for localizing messages for this logger. May be null if none of the messages require localization.

Returns:

a newly created private Logger

Throws:

MissingResourceException - if the resourceBundleName is non-null and no corresponding resource can be found.

getResourceBundle

```
public ResourceBundle getResourceBundle()
```

Retrieve the localization resource bundle for this logger. This method will return a ResourceBundle that was either set by the setResourceBundle method or mapped from the the resource bundle name set via the getLogger factory method for the current default locale.
Note that if the result is null, then the Logger will use a resource bundle or resource bundle name inherited from its parent.

Returns:

localization bundle (may be null)

getResourceBundleName

```
public String getResourceBundleName()
```

Retrieve the localization resource bundle name for this logger. This is either the name specified through the getLogger factory method, or the base name of the ResourceBundle set through setResourceBundle method.
Note that if the result is null, then the Logger will use a resource bundle or resource bundle name inherited from its parent.

Returns:

localization bundle name (may be null)

setFilter

```
public void setFilter(Filter newFilter)
              throws SecurityException
```

Set a filter to control output on this Logger.

After passing the initial "level" check, the Logger will call this Filter to check if a log record should really be published.

Parameters:

`newFilter - a filter object (may be null)`

Throws:

`SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").`

getFilter

```
public Filter getFilter()
```

Get the current filter for this Logger.

Returns:

`a filter object (may be null)`

log

```
public void log(LogRecord record)
```

Log a LogRecord.

All the other logging methods in this class call through this method to actually perform any logging. Subclasses can override this single method to capture all log activity.

Parameters:

`record - the LogRecord to be published`

log

```
public void log(Level level,
                String msg)
```

Log a message, with no arguments.

If the logger is currently enabled for the given message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`msg - The string message (or a key in the message catalog)`

log

```
public void log(Level level,
                Supplier<String> msgSupplier)
```

Log a message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the given message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

log

```
public void log(Level level,
                String msg,
                Object param1)
```

Log a message, with one object parameter.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`msg - The string message (or a key in the message catalog)`

`param1 - parameter to the message`

log

```
public void log(Level level,
                String msg,
                Object[] params)
```

Log a message, with an array of object arguments.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`msg - The string message (or a key in the message catalog)`

`params - array of parameters to the message`

log

```
public void log(Level level,
                String msg,
                Throwable thrown)
```

Log a message, with associated Throwable information.

If the logger is currently enabled for the given message level then the given arguments are stored in a LogRecord which is forwarded to all registered output handlers.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`msg - The string message (or a key in the message catalog)`

`thrown - Throwable associated with log message.`

log

```
public void log(Level level,
                Throwable thrown,
                Supplier<String> msgSupplier)
```

Log a lazily constructed message, with associated Throwable information.

If the logger is currently enabled for the given message level then the message is constructed by invoking the provided supplier function. The message and the given `Throwable` are then stored in a `LogRecord` which is forwarded to all registered output handlers.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed

specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`thrown - Throwable associated with log message.`

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 String msg)
```

Log a message, specifying source class and method, with no arguments.

If the logger is currently enabled for the given message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`msg - The string message (or a key in the message catalog)`

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 Supplier<String> msgSupplier)
```

Log a lazily constructed message, specifying source class and method, with no arguments.

If the logger is currently enabled for the given message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

sourceClass - name of class that issued the logging request

sourceMethod - name of method that issued the logging request

msgSupplier - A function, which when called, produces the desired log message

Since:

1.8

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 String msg,
                 Object param1)
```

Log a message, specifying source class and method, with a single object parameter to the log message.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

Parameters:

level - One of the message level identifiers, e.g., SEVERE

sourceClass - name of class that issued the logging request

sourceMethod - name of method that issued the logging request

msg - The string message (or a key in the message catalog)

param1 - Parameter to the log message.

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 String msg,
                 Object[] params)
```

Log a message, specifying source class and method, with an array of object arguments.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`msg - The string message (or a key in the message catalog)`

`params - Array of parameters to the message`

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 String msg,
                 Throwable thrown)
```

Log a message, specifying source class and method, with associated Throwable information.

If the logger is currently enabled for the given message level then the given arguments are stored in a LogRecord which is forwarded to all registered output handlers.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`msg - The string message (or a key in the message catalog)`

`thrown - Throwable associated with log message.`

logp

```
public void logp(Level level,
                 String sourceClass,
                 String sourceMethod,
                 Throwable thrown,
                 Supplier<String> msgSupplier)
```

Log a lazily constructed message, specifying source class and method, with associated Throwable information.

If the logger is currently enabled for the given message level then the message is constructed by invoking the provided supplier function. The message and the given `Throwable` are then stored in a `LogRecord` which is forwarded to all registered output handlers.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`thrown - Throwable associated with log message.`

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

logrb

```
@Deprecated
public void logrb(Level level,
                              String sourceClass,
                              String sourceMethod,
                              String bundleName,
                              String msg)
```

Deprecated. *Use `logrb(java.util.logging.Level, java.lang.String, java.lang.String, java.util.ResourceBundle, java.lang.String, java.lang.Object...)` instead.*

Log a message, specifying source class, method, and resource bundle name with no arguments.

If the logger is currently enabled for the given message level then the given message is forwarded to all the registered output Handler objects.

The msg string is localized using the named resource bundle. If the resource bundle name is null, or an empty String or invalid then the msg string is not localized.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

sourceClass - name of class that issued the logging request

sourceMethod - name of method that issued the logging request

bundleName - name of resource bundle to localize msg, can be null

msg - The string message (or a key in the message catalog)

logrb

```
@Deprecated
public void logrb(Level level,
                  String sourceClass,
                  String sourceMethod,
                  String bundleName,
                  String msg,
                  Object param1)
```

Deprecated. *Use `logrb(java.util.logging.Level, java.lang.String, java.lang.String, java.util.ResourceBundle, java.lang.String, java.lang.Object...)` instead*

Log a message, specifying source class, method, and resource bundle name, with a single object parameter to the log message.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

The msg string is localized using the named resource bundle. If the resource bundle name is null, or an empty String or invalid then the msg string is not localized.

Parameters:

level - One of the message level identifiers, e.g., SEVERE

sourceClass - name of class that issued the logging request

sourceMethod - name of method that issued the logging request

bundleName - name of resource bundle to localize msg, can be null

msg - The string message (or a key in the message catalog)

param1 - Parameter to the log message.

logrb

```
@Deprecated
public void logrb(Level level,
                  String sourceClass,
                  String sourceMethod,
```

```
                                String bundleName,
                                String msg,
                                Object[] params)
```

Deprecated. *Use `logrb(java.util.logging.Level, java.lang.String, java.lang.String, java.util.ResourceBundle, java.lang.String, java.lang.Object...)` instead.*

Log a message, specifying source class, method, and resource bundle name, with an array of object arguments.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

The msg string is localized using the named resource bundle. If the resource bundle name is null, or an empty String or invalid then the msg string is not localized.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`bundleName - name of resource bundle to localize msg, can be null.`

`msg - The string message (or a key in the message catalog)`

`params - Array of parameters to the message`

logrb

```
public void logrb(Level level,
                  String sourceClass,
                  String sourceMethod,
                  ResourceBundle bundle,
                  String msg,
                  Object... params)
```

Log a message, specifying source class, method, and resource bundle, with an optional list of message parameters.

If the logger is currently enabled for the given message level then a corresponding LogRecord is created and forwarded to all the registered output Handler objects.

The `msg` string is localized using the given resource bundle. If the resource bundle is `null`, then the `msg` string is not localized.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

sourceClass - Name of the class that issued the logging request

sourceMethod - Name of the method that issued the logging request

bundle - Resource bundle to localize msg, can be null.

msg - The string message (or a key in the message catalog)

params - Parameters to the message (optional, may be none).

Since:

1.8

logrb

```
@Deprecated
public void logrb(Level level,
                                String sourceClass,
                                String sourceMethod,
                                String bundleName,
                                String msg,
                                Throwable thrown)
```

Deprecated. *Use `logrb(java.util.logging.Level, java.lang.String, java.lang.String, java.util.ResourceBundle, java.lang.String, java.lang.Throwable)` instead.*

Log a message, specifying source class, method, and resource bundle name, with associated Throwable information.

If the logger is currently enabled for the given message level then the given arguments are stored in a LogRecord which is forwarded to all registered output handlers.

The msg string is localized using the named resource bundle. If the resource bundle name is null, or an empty String or invalid then the msg string is not localized.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that issued the logging request`

`bundleName - name of resource bundle to localize msg, can be null`

`msg - The string message (or a key in the message catalog)`

`thrown - Throwable associated with log message.`

logrb

```
public void logrb(Level level,
                  String sourceClass,
                  String sourceMethod,
                  ResourceBundle bundle,
                  String msg,
                  Throwable thrown)
```

Log a message, specifying source class, method, and resource bundle, with associated Throwable information.

If the logger is currently enabled for the given message level then the given arguments are stored in a LogRecord which is forwarded to all registered output handlers.

The `msg` string is localized using the given resource bundle. If the resource bundle is `null`, then the `msg` string is not localized.

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`level - One of the message level identifiers, e.g., SEVERE`

`sourceClass - Name of the class that issued the logging request`

`sourceMethod - Name of the method that issued the logging request`

`bundle - Resource bundle to localize msg, can be null`

`msg - The string message (or a key in the message catalog)`

`thrown - Throwable associated with the log message.`

Since:

`1.8`

entering

```
public void entering(String sourceClass,
                     String sourceMethod)
```

Log a method entry.

This is a convenience method that can be used to log entry to a method. A LogRecord with message "ENTRY", log level FINER, and the given sourceMethod and sourceClass is logged.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that is being entered`

entering

```
public void entering(String sourceClass,
                     String sourceMethod,
                     Object param1)
```

Log a method entry, with one parameter.

This is a convenience method that can be used to log entry to a method. A LogRecord with message "ENTRY {0}", log level FINER, and the given sourceMethod, sourceClass, and parameter is logged.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that is being entered`

`param1 - parameter to the method being entered`

entering

```
public void entering(String sourceClass,
                     String sourceMethod,
                     Object[] params)
```

Log a method entry, with an array of parameters.

This is a convenience method that can be used to log entry to a method. A LogRecord with message "ENTRY" (followed by a format {N} indicator for each entry in the parameter array), log level FINER, and the given sourceMethod, sourceClass, and parameters is logged.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of method that is being entered`

`params - array of parameters to the method being entered`

exiting

```
public void exiting(String sourceClass,
                    String sourceMethod)
```

Log a method return.

This is a convenience method that can be used to log returning from a method. A LogRecord with message "RETURN", log level FINER, and the given sourceMethod and sourceClass is logged.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of the method`

exiting

```
public void exiting(String sourceClass,
                    String sourceMethod,
                    Object result)
```

Log a method return, with result object.

This is a convenience method that can be used to log returning from a method. A LogRecord with message "RETURN {0}", log level FINER, and the gives sourceMethod, sourceClass, and result object is logged.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of the method`

`result - Object that is being returned`

throwing

```
public void throwing(String sourceClass,
                     String sourceMethod,
                     Throwable thrown)
```

Log throwing an exception.

This is a convenience method to log that a method is terminating by throwing an exception. The logging is done using the FINER level.

If the logger is currently enabled for the given message level then the given arguments are stored in a LogRecord which is forwarded to all registered output handlers. The LogRecord's message is set to "THROW".

Note that the thrown argument is stored in the LogRecord thrown property, rather than the LogRecord parameters property. Thus it is processed specially by output Formatters and is not treated as a formatting parameter to the LogRecord message property.

Parameters:

`sourceClass - name of class that issued the logging request`

`sourceMethod - name of the method.`

`thrown - The Throwable that is being thrown.`

severe

```
public void severe(String msg)
```

Log a SEVERE message.

If the logger is currently enabled for the SEVERE message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

`msg - The string message (or a key in the message catalog)`

warning

```
public void warning(String msg)
```

Log a WARNING message.

If the logger is currently enabled for the WARNING message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

msg - The string message (or a key in the message catalog)

info

```
public void info(String msg)
```

Log an INFO message.

If the logger is currently enabled for the INFO message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

msg - The string message (or a key in the message catalog)

config

```
public void config(String msg)
```

Log a CONFIG message.

If the logger is currently enabled for the CONFIG message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

msg - The string message (or a key in the message catalog)

fine

```
public void fine(String msg)
```

Log a FINE message.

If the logger is currently enabled for the FINE message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

msg - The string message (or a key in the message catalog)

finer

```
public void finer(String msg)
```

Log a FINER message.

If the logger is currently enabled for the FINER message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

`msg - The string message (or a key in the message catalog)`

finest

```
public void finest(String msg)
```

Log a FINEST message.

If the logger is currently enabled for the FINEST message level then the given message is forwarded to all the registered output Handler objects.

Parameters:

`msg - The string message (or a key in the message catalog)`

severe

```
public void severe(Supplier<String> msgSupplier)
```

Log a SEVERE message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the SEVERE message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

warning

```
public void warning(Supplier<String> msgSupplier)
```

Log a WARNING message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the WARNING message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

1.8

info

```
public void info(Supplier<String> msgSupplier)
```

Log a INFO message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the INFO message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

1.8

config

```
public void config(Supplier<String> msgSupplier)
```

Log a CONFIG message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the CONFIG message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

1.8

fine

```
public void fine(Supplier<String> msgSupplier)
```

Log a FINE message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the FINE message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

finer

```
public void finer(Supplier<String> msgSupplier)
```

Log a FINER message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the FINER message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

finest

```
public void finest(Supplier<String> msgSupplier)
```

Log a FINEST message, which is only to be constructed if the logging level is such that the message will actually be logged.

If the logger is currently enabled for the FINEST message level then the message is constructed by invoking the provided supplier function and forwarded to all the registered output Handler objects.

Parameters:

`msgSupplier - A function, which when called, produces the desired log message`

Since:

`1.8`

setLevel

```
public void setLevel(Level newLevel)
              throws SecurityException
```

Set the log level specifying which message levels will be logged by this logger. Message levels lower than this value will be discarded. The level value Level.OFF can be used to turn off logging.

If the new level is null, it means that this node should inherit its level from its nearest ancestor with a specific (non-null) level value.

Parameters:

`newLevel - the new value for the log level (may be null)`

Throws:

`SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").`

getLevel

```
public Level getLevel()
```

Get the log Level that has been specified for this Logger. The result may be null, which means that this logger's effective level will be inherited from its parent.

Returns:

`this Logger's level`

isLoggable

```
public boolean isLoggable(Level level)
```

Check if a message of the given level would actually be logged by this logger. This check is based on the Loggers effective level, which may be inherited from its parent.

Parameters:

`level - a message logging level`

Returns:

`true if the given message level is currently being logged.`

getName

```
public String getName()
```

Get the name for this logger.

Returns:

logger name. Will be null for anonymous Loggers.

addHandler

```
public void addHandler(Handler handler)
                throws SecurityException
```

Add a log Handler to receive logging messages.

By default, Loggers also send their output to their parent logger. Typically the root Logger is configured with a set of Handlers that essentially act as default handlers for all loggers.

Parameters:

handler - a logging Handler

Throws:

SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").

removeHandler

```
public void removeHandler(Handler handler)
                  throws SecurityException
```

Remove a log Handler.

Returns silently if the given Handler is not found or is null

Parameters:

handler - a logging Handler

Throws:

SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").

getHandlers

```
public Handler[] getHandlers()
```

Get the Handlers associated with this logger.

Returns:

an array of all registered Handlers

setUseParentHandlers

```
public void setUseParentHandlers(boolean useParentHandlers)
```

Specify whether or not this logger should send its output to its parent Logger. This means that any LogRecords will also be written to the parent's Handlers, and potentially to its parent, recursively up the namespace.

Parameters:

useParentHandlers - true if output is to be sent to the logger's parent.

Throws:

SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").

getUseParentHandlers

```
public boolean getUseParentHandlers()
```

Discover whether or not this logger is sending its output to its parent logger.

Returns:

true if output is to be sent to the logger's parent

setResourceBundle

```
public void setResourceBundle(ResourceBundle bundle)
```

Sets a resource bundle on this logger. All messages will be logged using the given resource bundle for its specific locale.

Parameters:

bundle - The resource bundle that this logger shall use.

Throws:

NullPointerException - if the given bundle is null.

IllegalArgumentException - if the given bundle doesn't have a base name, or if this logger already has a resource bundle set but the given bundle has a different base name.

SecurityException - if a security manager exists, this logger is not anonymous, and the caller does not have LoggingPermission("control").

Since:

1.8

getParent

```
public Logger getParent()
```

Return the parent for this Logger.

This method returns the nearest extant parent in the namespace. Thus if a Logger is called "a.b.c.d", and a Logger called "a.b" has been created but no logger "a.b.c" exists, then a call of getParent on the Logger "a.b.c.d" will return the Logger "a.b".

The result will be null if it is called on the root Logger in the namespace.

Returns:

nearest existing parent Logger

setParent

```
public void setParent(Logger parent)
```

Set the parent for this Logger. This method is used by the LogManager to update a Logger when the namespace changes.

It should not be called from application code.

Parameters:

parent - the new parent logger

Throws:

```
SecurityException - if a security manager exists and if the caller
does not have LoggingPermission("control").
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.